IGWT Lectures

WebGL and Shader Snippets

Catalogs of pipeline, lighting, noise, SDF, postprocess, and GPU snippets. Open WebGL/demos for runnable HTML.

07 WebGL and Shader Snippets.md

07 WebGL and Shader Snippets

Runnable WebGL2 demos and copy-paste GLSL for **IGWT** Courses 7, 10, 11, and 12.

These are teaching snippets, not a production engine. Each demo is one idea. Open the HTML files in a browser (a local static server is safest). Do not start from Three.js here. Three.js comes after students can get a triangle on screen from this folder.

Start

1. 00 Index — map of demos and catalogs
2. 01 Conventions — clip space, winding, color, black-screen checklist
3. Open `WebGL/demos/01-triangle.html`

What is in the folder

Path	What it is
00 Index	Catalog of every demo and GLSL file
01 Conventions	Spaces, winding, gamma, debug
02 JS Helpers	Shared `demos/_gl.js` documented
10 Pipeline	Context, shaders, buffers, VAO, uniforms, textures, FBO
11 Vertex and Fragment	Attribute plumbing, varyings, clip-space patterns
12 Lighting	Lambert, Phong, Blinn, PBR, toon, fresnel, IBL-lite
13 Noise	Hash, value noise, fbm, voronoi, warp
14 SDF and Ray Marching	Primitives, CSG, sphere tracer
15 Postprocess	Fullscreen pass, bloom-lite, FX, color
16 Effects	Fog, dissolve, water, fire, triplanar, matcap, barycentric
17 Particles and GPGPU	Points, ping-pong, instancing
	Self-contained HTML you can open and edit

Teaching use

- Course 7: demos 01–08, then 10 Pipeline
- Course 10: demos 09–12, 16, 20–21 and 13 Noise, 14 SDF and Ray Marching

- Course 11: demos 13, 16, 22 and 12 Lighting, 15 Postprocess
- Course 12: demos 14–15, 23 and 17 Particles and GPGPU

Live-code from a demo, then delete a function and have students restore it. See 06 Live Coding Pedagogy.

License

00 Index.md

WebGL snippet index

Parent: 07 WebGL and Shader Snippets

Open files under `WebGL/demos/`. They load `demos/_gl.js` with a classic `<script>` tag (works from `file://` on most browsers). If a texture or module fails, run `npx serve WebGL/demos`.

Demos (open these)

File	Idea	Course
`demos/01-triangle.html`	Smallest WebGL2 program	7
`demos/02-colored-triangle.html`	Per-vertex colors as attributes	7
`demos/03-indexed-quad.html`	`ELEMENT_ARRAY_BUFFER`	7
`demos/04-rotating-cube.html`	Depth test, cull, MVP	7
`demos/05-canvas-texture.html`	Texture from a canvas (no CORS)	7
`demos/06-phong-cube.html`	Normals + Blinn-Phong	7 / 10
`demos/07-orbit-camera.html`	lookAt + mouse orbit	7
`demos/08-uv-debug.html`	UV as color, checker	7
`demos/09-fullscreen-gradient.html`	Fullscreen triangle, `gl_FragCoord`	10
`demos/10-noise-fbm.html`	Hash, fbm, domain warp	10
`demos/11-sdf-raymarch.html`	Sphere + box, soft shadow	10
`demos/12-toon-fresnel.html`	Quantized diffuse + rim	10
`demos/13-framebuffer-post.html`	Render to texture, vignette	11 / 12
`demos/14-instancing.html`	`drawArraysInstanced`	12
`demos/15-particles.html`	`gl.POINTS`, size, fade	12
`demos/16-pbr-direct.html`	Cook–Torrance, one light	11
`demos/17-normal-colors.html`	Debug: world normal as RGB	7
`demos/18-barycentric-wire.html`	Wireframe without a second mesh	10
`demos/19-dissolve.html`	Noise cutoff + edge color	10
`demos/20-water.html`	Sum of sines, fake spec	10
`demos/21-sky.html`	Gradient sky + sun disc	10
`demos/22-shadow-map.html`	Depth FBO, compare in light space	11
`demos/23-gpgpu-pingpong.html`	Two textures, particle integrate	12
`demos/24-matcap.html`	View-space normal → matcap UV	10
`demos/25-webgl1-triangle.html`	Same triangle in WebGL1 / GLSL 100	7

GLSL / JS catalogs (copy-paste)

Note	Contents
01 Conventions	Spaces, winding, gamma, checklist
02 JS Helpers	Compile, matrices, geometry, FBO
10 Pipeline	Context through framebuffer
11 Vertex and Fragment	Shader skeletons
12 Lighting	Lambert → PBR
13 Noise	Hash through voronoi
14 SDF and Ray Marching	SDF ops + tracer
15 Postprocess	Fullscreen FX
16 Effects	Fog, fire, triplanar, …
17 Particles and GPGPU	Points, instancing, ping-pong

How to use in a lab

1. Open one demo.
2. Change one number. Predict the picture. Then change it.
3. Break it (comment out `enable(DEPTH_TEST)`). Use the checklist.
4. Copy one GLSL function from a catalog into the demo.

Do not paste five catalogs into week 1. Triangle first.

01 Conventions.md

01 — Conventions

Parent: 07 WebGL and Shader Snippets

Disagreeing about conventions is the main source of "my cube is inside out." Freeze these for a term.

Spaces

```
object (model) → world → view (camera) → clip → NDC → framebuffer pixels
```

Space	Who	Typical names
Object	Mesh verts	`a_position`
World	After model matrix	`v_worldPos`
View	Camera at origin, looking −Z	`u_view * world`
Clip	After projection, before divide	`gl_Position`
NDC	After perspective divide, xyz in [−1,1]	GPU
Pixel	`gl_FragCoord.xy`	Fragment shader

This program: **right-handed, Y-up, camera looks down −Z**, **column-major** `mat4` in the order `proj * view * model * vec4(p,1)`.

Normals: `mat3(transpose(inverse(model)))` if the model has non-uniform scale. For uniform scale, `mat3(model)` is enough.

Winding and culling

- Vertex order **counter-clockwise** is front (OpenGL default).
- `gl.cullFace(gl.BACK)` after `enable(CULL_FACE)`.
- If a model from Blender looks inside-out, the export winding or the `frontFace` setting is wrong — not "WebGL is broken."

Clip and depth

- `gl_Position.w` is not 1 after perspective. Do not divide in the vertex shader; the GPU does it.
- Depth range is [0,1] in the framebuffer. `enable(DEPTH_TEST)` and `depthFunc(LEQUAL)` are the usual pair.
- Near plane too close → z-fighting. Start with near `0.1`, far `100`.

Color

- Textures from canvases and most PNGs are **sRGB**. Sampling as linear vs sRGB changes lighting.
- Do lighting in **linear** space. Apply `pow(color, vec3(1.0/2.2))` at the end of the fragment shader unless you use `EXT_sRGB` / WebGL2 sRGB framebuffers.
- Never encode "category" in color alone in a teaching figure; also label. See 10 Inclusive Teaching and Accessibility.

GLSL versions

API	Shader first line
WebGL1	`#ifdef GL_ES\nprecision highp float;\n#endif` (GLSL ES 1.00)
WebGL2	`#version 300 es` then `precision highp float;`

WebGL2: `attribute` → `in`, `varying` → `in` / `out`, `gl_FragColor` → you declare `out vec4 outColor`, `texture2D` → `texture`.

Black-screen checklist

Same list as 06 Live Coding Pedagogy:

1. Canvas in the DOM and sized (not 0×0 CSS / backing store)
2. Clear color you would notice (`0.1, 0.1, 0.12, 1`)
3. Shader compile and **link** logs
4. Camera looks at the object
5. Object in front of the near plane
6. Winding / culling
7. Depth test / write
8. Attributes bound to the locations you used
9. Texture finished uploading (async)
10. Color / alpha making it invisible (premultiplied, `discard`, alpha 0)

Debug shaders (keep these)

Normal as color:

```
outColor = vec4(normalize(v_normal) * 0.5 + 0.5, 1.0);
```

UV as color:

```
outColor = vec4(v_uv, 0.0, 1.0);
```

Depth as gray (after you linearize, or raw for a quick look):

```
outColor = vec4(vec3(gl_FragCoord.z), 1.0);
```

02 JS Helpers.md

02 — JS helpers (`demos/_gl.js`)

Parent: 07 WebGL and Shader Snippets

The file `WebGL/demos/_gl.js` is a small teaching runtime. Students should **read it**, not treat it as magic. After Course 7 week 3 they should be able to rewrite `program()` from memory.

Create context and resize

```
const gl = GL.createGL(canvas);
function frame() {
  const aspect = GL.resize(gl); // sets canvas backing store + viewport
}
```

`resize` uses `clientWidth/Height` and a capped DPR. If the canvas CSS size is 0, you get a black screen — checklist item 1.

Compile and link (always print the log)

```
const prog = GL.program(gl, vsSource, fsSource, ["a_position", "a_normal", "a_uv"]);
const u = GL.uniforms(gl, prog);
```

`bindAttribLocation` before link keeps locations stable (0, 1, 2, …). After link, `GL.uniforms` maps names to locations (array uniforms drop `[0]`).

Raw form, for lectures without the helper:

```
function compile(gl, type, src) {
  const sh = gl.createShader(type);
  gl.shaderSource(sh, src);
  gl.compileShader(sh);
  if (!gl.getShaderParameter(sh, gl.COMPILE_STATUS)) {
    throw new Error(gl.getShaderInfoLog(sh));
  }
  return sh;
}
```

Buffers and VAO

```
const vao = GL.vao(gl, () => {
  GL.buffer(gl, mesh.pos);
  GL.enableAttrib(gl, 0, 3);
  GL.buffer(gl, mesh.nor);
  GL.enableAttrib(gl, 1, 3);
  GL.buffer(gl, mesh.uv);
  GL.enableAttrib(gl, 2, 2);
  GL.buffer(gl, mesh.idx, gl.ELEMENT_ARRAY_BUFFER);
});
gl.bindVertexArray(vao);
gl.drawElements(gl.TRIANGLES, mesh.count, gl.UNSIGNED_SHORT, 0);
```

WebGL2 VAOs store the element buffer binding. Re-bind the VAO before drawing.

Matrices

```
const proj = GL.mat4Perspective(Math.PI / 3, aspect, 0.1, 100);
const view = GL.mat4LookAt(eye, [0, 0, 0], [0, 1, 0]);
const model = GL.mat4Mul(GL.mat4Ry(t), GL.mat4Rx(0.4));
const nmat = GL.mat3Normal(model);
gl.uniformMatrix4fv(u.u_proj, false, proj);
gl.uniformMatrix4fv(u.u_view, false, view);
gl.uniformMatrix4fv(u.u_model, false, model);
gl.uniformMatrix3fv(u.u_normal, false, nmat);
```

`false` means “already column-major.” Do not transpose twice.

Geometry

- `GL.cube()` — 24 unique verts (normals per face), indexed
- `GL.sphere(lat, lon)` — unit sphere
- `GL.fullscreenVerts()` — one oversized triangle covering the screen

Textures without files

```
const tex = GL.checkerTexture(gl, 8);
gl.activeTexture(gl.TEXTURE0);
gl.bindTexture(gl.TEXTURE_2D, tex);
gl.uniform1i(u.u_tex, 0);
```

A 1×1 `GL.colorTexture(gl, [255, 80, 40])` is a valid albedo while you debug lighting.

Framebuffers

```
const rt = GL.fbo(gl, 512, 512, true);      // color + depth renderbuffer
const sh = GL.depthFbo(gl, 1024, 1024);     // depth texture for shadows
```

After drawing to `rt.fb`, bind `null` and sample `rt.color`.

Orbit camera and loop

```
const cam = GL.orbit(canvas, { dist: 4 });
GL.loop((dt, t) => {
  const eye = cam.eye();
  const view = GL.mat4LookAt(eye, cam.target, [0, 1, 0]);
});
```

Drag to orbit, wheel to dolly. `dt` is seconds since last frame; `t` is seconds since start.

What not to add to this file

Instancing setup, skinning, glTF, and WebGPU. Those belong in a later demo or a real engine. Keep `_gl.js` small enough to read in one lab.

10 Pipeline.md

10 — Pipeline snippets

Parent: 07 WebGL and Shader Snippets

Copy these into a blank HTML file during Course 7. After week 4, students should not need this page open.

Canvas + context

```
<canvas id="c" style="width:100%;height:100%;display:block"></canvas>
<script>
const canvas = document.getElementById("c");
const gl = canvas.getContext("webgl2", { antialias: true, alpha: false });
if (!gl) throw new Error("Need WebGL2");
gl.clearColor(0.10, 0.10, 0.12, 1);
gl.enable(gl.DEPTH_TEST);
gl.enable(gl.CULL_FACE);
</script>
```

Shader compile / link with logs

```
function makeProgram(gl, vsSrc, fsSrc) {
  function sh(type, src) {
    const s = gl.createShader(type);
    gl.shaderSource(s, src);
    gl.compileShader(s);
    if (!gl.getShaderParameter(s, gl.COMPILE_STATUS)) {
      console.error(gl.getShaderInfoLog(s));
      throw new Error("compile");
    }
    return s;
  }
  const p = gl.createProgram();
  gl.attachShader(p, sh(gl.VERTEX_SHADER, vsSrc));
  gl.attachShader(p, sh(gl.FRAGMENT_SHADER, fsSrc));
  gl.linkProgram(p);
  if (!gl.getProgramParameter(p, gl.LINK_STATUS)) {
    console.error(gl.getProgramInfoLog(p));
    throw new Error("link");
  }
  return p;
}
```

A **link** error is not a compile error. Students often only check compile.

Interleaved vs separate buffers

Separate (clearer for teaching):

```
gl.bindBuffer(gl.ARRAY_BUFFER, posBuf);
gl.enableVertexAttribArray(0);
gl.vertexAttribPointer(0, 3, gl.FLOAT, false, 0, 0);
gl.bindBuffer(gl.ARRAY_BUFFER, nrmBuf);
gl.enableVertexAttribArray(1);
gl.vertexAttribPointer(1, 3, gl.FLOAT, false, 0, 0);
```

Interleaved `px,py,pz,nx,ny,nz,u,v` (32 bytes):

```
const stride = 32;
gl.vertexAttribPointer(0, 3, gl.FLOAT, false, stride, 0);
gl.vertexAttribPointer(1, 3, gl.FLOAT, false, stride, 12);
gl.vertexAttribPointer(2, 2, gl.FLOAT, false, stride, 24);
```

Indexed drawing

```
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, idxBuf);
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, indices, gl.STATIC_DRAW);
gl.drawElements(gl.TRIANGLES, indices.length, gl.UNSIGNED_SHORT, 0);
```

Use `UNSIGNED_INT` + `OES_element_index_uint` (WebGL1) or WebGL2 native for meshes > 65535 verts.

Uniforms

```
gl.uniform1f(loc, t);
gl.uniform2f(loc, w, h);
gl.uniform3fv(loc, new Float32Array([0, 1, 0]));
gl.uniform4f(loc, r, g, b, a);
gl.uniformMatrix4fv(loc, false, mat4);
gl.uniform1i(samplerLoc, 0); // texture unit, not a boolean
```

`uniform1i` for samplers. Forgetting this leaves the sampler on unit 0 by luck — until you bind two textures.

Texture upload

```
const tex = gl.createTexture();
gl.bindTexture(gl.TEXTURE_2D, tex);
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, 1);
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, image);
gl.generateMipmap(gl.TEXTURE_2D);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
```

NPOT wrap with `REPEAT` is fine in WebGL2. In WebGL1 NPOT cannot mipmap or REPEAT.

Async image:

```
const img = new Image();
img.onload = () => { /* texImage2D */ };
img.src = "albedo.png";
```

Until `onload`, bind a 1×1 color or you sample garbage.

Texture units

```
gl.activeTexture(gl.TEXTURE0);
gl.bindTexture(gl.TEXTURE_2D, albedo);
gl.uniform1i(u.u_albedo, 0);
gl.activeTexture(gl.TEXTURE1);
gl.bindTexture(gl.TEXTURE_2D, normalMap);
gl.uniform1i(u.u_normal, 1);
```

Render to texture

```
gl.bindFramebuffer(gl.FRAMEBUFFER, rt.fb);
gl.viewport(0, 0, rt.w, rt.h);
gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
// draw scene
gl.bindFramebuffer(gl.FRAMEBUFFER, null);
gl.viewport(0, 0, gl.canvas.width, gl.canvas.height);
```

Readback (slow, debug only)

```
const px = new Uint8Array(4);
gl.readPixels(x, y, 1, 1, gl.RGBA, gl.UNSIGNED_BYTE, px);
```

Origin is **lower-left** in `readPixels`, upper-left in CSS. Flip `y`.

State you will forget to reset

State	Default trap
`blend`	Left on after a transparent pass
`depthMask`	False after particles, then the next opaque fails
`viewport`	Still the FBO size
`activeTexture`	Not 0
VAO	Still bound; next buffer upload hits the wrong VAO

Reset at the start of a frame if you are lost:

```
gl.bindVertexArray(null);
gl.bindFramebuffer(gl.FRAMEBUFFER, null);
gl.disable(gl.BLEND);
gl.depthMask(true);
gl.enable(gl.DEPTH_TEST);
gl.enable(gl.CULL_FACE);
```

WebGL1 vs WebGL2 cheatsheet

Task	WebGL1	WebGL2
VAO	`OES_vertex_array_object`	native
Instancing	`ANGLE_instanced_arrays`	native
GLSL	100 es	300 es
Integer elements	extension	native
Depth texture	`WEBGL_depth_texture`	native
MRT	`WEBGL_draw_buffers`	`drawBuffers`

11 Vertex and Fragment.md

11 — Vertex and fragment patterns

Parent: 07 WebGL and Shader Snippets

All snippets are **GLSL ES 3.00** (`#version 300 es`) unless noted.

Minimal pair (clip-space triangle)

```
#version 300 es
in vec2 a_position;
void main() {
  gl_Position = vec4(a_position, 0.0, 1.0);
}
```

```
#version 300 es
precision highp float;
out vec4 outColor;
void main() {
  outColor = vec4(0.91, 0.31, 0.22, 1.0);
}
```

`a_position` in [−1,1] is already NDC if `z=0,w=1`.

Pass-through MVP + world position / normal / UV

```
#version 300 es
layout(location = 0) in vec3 a_position;
layout(location = 1) in vec3 a_normal;
layout(location = 2) in vec2 a_uv;
uniform mat4 u_proj, u_view, u_model;
uniform mat3 u_normal;
out vec3 v_worldPos;
out vec3 v_worldN;
out vec2 v_uv;
void main() {
  vec4 w = u_model * vec4(a_position, 1.0);
  v_worldPos = w.xyz;
  v_worldN = normalize(u_normal * a_normal);
  v_uv = a_uv;
  gl_Position = u_proj * u_view * w;
}
```

View-space position (fog, lighting)

```
vec3 viewPos = (u_view * vec4(v_worldPos, 1.0)).xyz;
```

Or compute in the vertex shader and pass `v_viewPos`.

Billboard (face camera, keep world position)

```
vec3 camRight = vec3(u_view[0][0], u_view[1][0], u_view[2][0]);
vec3 camUp    = vec3(u_view[0][1], u_view[1][1], u_view[2][1]);
vec3 world = a_center + camRight * a_position.x * a_size + camUp * a_position.y * a_size;
gl_Position = u_proj * u_view * vec4(world, 1.0);
```

`a_position` is the quad corner in [−0.5,0.5].

Point sprites

```
gl_PointSize = u_size * (u_scale / gl_Position.w);
```

Fragment:

```
vec2 p = gl_PointCoord * 2.0 - 1.0;
if (dot(p, p) > 1.0) discard;
```

`gl_PointCoord` origin is upper-left. Soft circle: `1.0 - smoothstep(0.7, 1.0, length(p))`.

Skinned vertex (two bones, teaching size)

```
in vec4 a_joints;  // as float indices
in vec4 a_weights;
uniform mat4 u_bones[32];
mat4 skin =
  a_weights.x * u_bones[int(a_joints.x)] +
  a_weights.y * u_bones[int(a_joints.y)] +
  a_weights.z * u_bones[int(a_joints.z)] +
  a_weights.w * u_bones[int(a_joints.w)];
vec4 world = u_model * skin * vec4(a_position, 1.0);
```

Normalize weights if they do not sum to 1. Do not start Course 7 here.

Derivative-based wireframe (no barycentric VBO)

```
float edge = min(min(v_bary.x, v_bary.y), v_bary.z);
float w = fwidth(edge);
float line = 1.0 - smoothstep(0.0, w * 1.2, edge);
```

Needs `v_bary` from the CPU (see demo 18) **or** `GL_NV_fragment_shader_barycentric` (not on the web). `fwidth` is a fragment-only call.

Clip / discard

```
if (v_worldPos.y < u_clipY) discard;
```

`discard` kills early-z on many GPUs. Prefer a clip plane in the vertex shader for opaque meshes:

```
gl_ClipDistance[0] = v_worldPos.y - u_clipY;
```

Requires `gl.enable(gl.CLIP_DISTANCE0)` (WebGL2).

Gamma helpers

```
vec3 toLinear(vec3 c) { return pow(c, vec3(2.2)); }
vec3 toSRGB(vec3 c)   { return pow(c, vec3(1.0 / 2.2)); }
```

Apply `toLinear` to sampled albedo if the texture is sRGB and the format is unsigned-byte RGBA. Apply `toSRGB` once at the end.

Precision

Use `highp` for positions and ray marching. `mediump` on fragment is enough for albedo tints on mobile, but it will break a long ray march.

WebGL1 pair (demo 25)

```
attribute vec2 a_position;
void main() {
  gl_Position = vec4(a_position, 0.0, 1.0);
}
```

```
precision mediump float;
void main() {
  gl_FragColor = vec4(0.91, 0.31, 0.22, 1.0);
}
```

12 Lighting.md

12 — Lighting snippets

Parent: 07 WebGL and Shader Snippets

Assume `N`, `V`, `L` are **normalized**. `N` world or view, as long as they match. `L` points **toward the light**. `H = normalize(L + V)`.

Lambert (diffuse)

```
float ndotl = max(dot(N, L), 0.0);
vec3 color = albedo * (u_ambient + u_lightColor * ndotl);
```

Hemisphere ambient (cheap sky/ground):

```
vec3 hemi = mix(u_ground, u_sky, N.y * 0.5 + 0.5);
vec3 color = albedo * hemi + albedo * u_lightColor * ndotl;
```

Phong specular

```
vec3 R = reflect(-L, N);
float spec = pow(max(dot(R, V), 0.0), u_shininess);
vec3 color = albedo * ndotl * u_lightColor + u_specColor * spec * u_lightColor;
```

`u_shininess` 8–128. Phong blows out if you add spec without `ndotl` gating; multiply spec by `step(0.0, ndotl)` so the back face is dark.

Blinn-Phong (preferred in class)

```
vec3 H = normalize(L + V);
float spec = pow(max(dot(N, H), 0.0), u_shininess);
```

Same energy caveats. Demo: `06-phong-cube.html`.

Attenuation (point light)

```
float d = length(lightPos - worldPos);
vec3 L = (lightPos - worldPos) / max(d, 1e-5);
float att = 1.0 / (u_c + u_l * d + u_q * d * d);
```

Start with `c=1, l=0, q=0.1` or a smooth window:

```
float att = clamp(1.0 - d / u_radius, 0.0, 1.0);
att *= att;
```

Toon / ramp

```
float d = max(dot(N, L), 0.0);
float bands = 4.0;
float toon = floor(d * bands) / bands;
vec3 color = albedo * mix(u_ambient, u_lightColor, toon);
```

Add a specular tick:

```
float spec = step(0.92, pow(max(dot(N, H), 0.0), 32.0));
```

Demo: `12-toon-fresnel.html`.

Fresnel (Schlick)

```
vec3 F0 = vec3(0.04); // dielectric
vec3 F = F0 + (1.0 - F0) * pow(1.0 - max(dot(N, V), 0.0), 5.0);
```

Rim / shell:

```
float rim = pow(1.0 - max(dot(N, V), 0.0), u_rimPow);
vec3 color += u_rimColor * rim;
```

Matcap (view-space normal)

```
vec3 vn = normalize(mat3(u_view) * N);
vec2 uv = vn.xy * 0.5 + 0.5;
vec3 color = texture(u_matcap, uv).rgb;
```

Demo: `24-matcap.html`. Generate a matcap as a procedural sphere lighting if you have no file.

Normal mapping (tangent space, TBN in fragment)

Vertex: pass `v_T, v_B, v_N` (orthonormalize). Fragment:

```
vec3 nTex = texture(u_normalMap, v_uv).xyz * 2.0 - 1.0;
nTex.xy *= u_normalScale;
vec3 N = normalize(mat3(normalize(v_T), normalize(v_B), normalize(v_N)) * nTex);
```

If you only have a normal map and UV derivatives:

```
vec3 nTex = texture(u_normalMap, v_uv).xyz * 2.0 - 1.0;
vec3 dp1 = dFdx(v_worldPos);
vec3 dp2 = dFdy(v_worldPos);
vec2 du1 = dFdx(v_uv);
vec2 du2 = dFdy(v_uv);
vec3 T = normalize(dp1 * du2.y - dp2 * du1.y);
vec3 B = normalize(-dp1 * du2.x + dp2 * du1.x);
vec3 N = normalize(mat3(T, B, normalize(v_worldN)) * nTex);
```

Cook–Torrance PBR (direct light, teaching form)

```
const float PI = 3.14159265;

float D_GGX(float NoH, float a) {
  float a2 = a * a;
  float d = NoH * NoH * (a2 - 1.0) + 1.0;
  return a2 / (PI * d * d);
}

float G_Smith(float NoV, float NoL, float a) {
  float k = (a + 1.0) * (a + 1.0) / 8.0;
  float gv = NoV / (NoV * (1.0 - k) + k);
  float gl = NoL / (NoL * (1.0 - k) + k);
  return gv * gl;
}

vec3 F_Schlick(float VoH, vec3 F0) {
  return F0 + (1.0 - F0) * pow(1.0 - VoH, 5.0);
}

vec3 pbr(vec3 albedo, float metallic, float roughness, vec3 N, vec3 V, vec3 L, vec3 lightColo
  float a = max(roughness * roughness, 0.002);
  vec3 H = normalize(V + L);
  float NoV = max(dot(N, V), 1e-4);
  float NoL = max(dot(N, L), 0.0);
  float NoH = max(dot(N, H), 0.0);
  float VoH = max(dot(V, H), 0.0);
  vec3 F0 = mix(vec3(0.04), albedo, metallic);
  vec3 F = F_Schlick(VoH, F0);
  float D = D_GGX(NoH, a);
  float G = G_Smith(NoV, NoL, roughness);
  vec3 spec = (D * G * F) / max(4.0 * NoV * NoL, 1e-4);
  vec3 kD = (1.0 - F) * (1.0 - metallic);
  vec3 diffuse = kD * albedo / PI;
  return (diffuse + spec) * lightColor * NoL;
}
```

Demo: `16-pbr-direct.html`. This is **one punctual light**. IBL is a later week.

Image-based ambient (fake)

```
vec3 ibl = mix(u_ground, u_sky, N.y * 0.5 + 0.5);
vec3 F = F0 + (1.0 - F0) * pow(1.0 - NoV, 5.0);
vec3 ambient = ibl * albedo * (1.0 - metallic) + ibl * F * (1.0 - roughness);
```

Not energy-conserving. Good enough to kill flat black creases.

Fog

```
float z = length(v_viewPos);
float fog = clamp((u_fogFar - z) / (u_fogFar - u_fogNear), 0.0, 1.0);
color = mix(u_fogColor, color, fog);
```

Height fog:

```
float h = smoothstep(u_fogY0, u_fogY1, v_worldPos.y);
color = mix(color, u_fogColor, (1.0 - fog) * (1.0 - h));
```

13 Noise.md

13 — Noise snippets

Parent: 07 WebGL and Shader Snippets

Noise in a fragment shader is a **hash of coordinates**, not `Math.random()`. These functions are deterministic so they match across pixels and frames.

Warn students before flashing high-contrast noise (10 Inclusive Teaching and Accessibility).

Integer hash → [0,1]

```
float hash11(float p) {
  p = fract(p * 0.1031);
  p *= p + 33.33;
  p *= p + p;
  return fract(p);
}

float hash12(vec2 p) {
  vec3 p3 = fract(vec3(p.xyx) * 0.1031);
  p3 += dot(p3, p3.yzx + 33.33);
  return fract((p3.x + p3.y) * p3.z);
}

vec2 hash22(vec2 p) {
  vec3 p3 = fract(vec3(p.xyx) * vec3(0.1031, 0.1030, 0.0973));
  p3 += dot(p3, p3.yzx + 33.33);
  return fract((p3.xx + p3.yz) * p3.zy);
}

vec3 hash33(vec3 p) {
  p = fract(p * vec3(0.1031, 0.1030, 0.0973));
  p += dot(p, p.yxz + 33.33);
  return fract((p.xxy + p.yxx) * p.zyx);
}
```

Value noise (2D)

```
float valueNoise(vec2 p) {
  vec2 i = floor(p);
  vec2 f = fract(p);
  vec2 u = f * f * (3.0 - 2.0 * f);
  float a = hash12(i);
  float b = hash12(i + vec2(1.0, 0.0));
  float c = hash12(i + vec2(0.0, 1.0));
  float d = hash12(i + vec2(1.0, 1.0));
  return mix(mix(a, b, u.x), mix(c, d, u.x), u.y);
}
```

Gradient noise (2D, Perlin-style)

```
float gradNoise(vec2 p) {
  vec2 i = floor(p);
  vec2 f = fract(p);
  vec2 u = f * f * (3.0 - 2.0 * f);
  float n00 = dot(hash22(i) * 2.0 - 1.0, f);
  float n10 = dot(hash22(i + vec2(1.0, 0.0)) * 2.0 - 1.0, f - vec2(1.0, 0.0));
  float n01 = dot(hash22(i + vec2(0.0, 1.0)) * 2.0 - 1.0, f - vec2(0.0, 1.0));
  float n11 = dot(hash22(i + vec2(1.0, 1.0)) * 2.0 - 1.0, f - vec2(1.0, 1.0));
  return mix(mix(n00, n10, u.x), mix(n01, n11, u.x), u.y);
}
```

fBm

```
float fbm(vec2 p) {
  float v = 0.0;
  float a = 0.5;
  mat2 m = mat2(1.6, 1.2, -1.2, 1.6);
  for (int i = 0; i < 5; i++) {
    v += a * valueNoise(p);
    p = m * p;
    a *= 0.5;
  }
  return v;
}
```

Use `gradNoise` for terrain; value noise is cheaper and blotchier.

Domain warp

```
vec2 q = vec2(fbm(uv), fbm(uv + 4.1));
float n = fbm(uv + 4.0 * q);
```

Demo: `10-noise-fbm.html`.

3D value noise (volume / fire)

```
float valueNoise3(vec3 p) {
  vec3 i = floor(p);
  vec3 f = fract(p);
  vec3 u = f * f * (3.0 - 2.0 * f);
  float n000 = hash13(i);
  float n100 = hash13(i + vec3(1,0,0));
  float n010 = hash13(i + vec3(0,1,0));
  float n110 = hash13(i + vec3(1,1,0));
  float n001 = hash13(i + vec3(0,0,1));
  float n101 = hash13(i + vec3(1,0,1));
  float n011 = hash13(i + vec3(0,1,1));
  float n111 = hash13(i + vec3(1,1,1));
  return mix(
    mix(mix(n000, n100, u.x), mix(n010, n110, u.x), u.y),
    mix(mix(n001, n101, u.x), mix(n011, n111, u.x), u.y),
    u.z);
}

float hash13(vec3 p) {
  p = fract(p * 0.1031);
  p += dot(p, p.zyx + 31.32);
  return fract((p.x + p.y) * p.z);
}
```

Voronoi / cellular

```
vec2 voronoi(vec2 p) {
  vec2 n = floor(p);
  vec2 f = fract(p);
  float md = 8.0;
  vec2 mr;
  for (int j = -1; j <= 1; j++)
  for (int i = -1; i <= 1; i++) {
    vec2 g = vec2(float(i), float(j));
    vec2 o = hash22(n + g);
    vec2 r = g + o - f;
    float d = dot(r, r);
    if (d < md) { md = d; mr = r; }
  }
  return vec2(sqrt(md), hash12(n + mr));
}
```

`x` is distance to nearest site; `y` is an id. Good for tiles, cracked earth, cells.

Tileable 2D (repeat period `rep`)

```
float hashTile(vec2 p, float rep) {
  return hash12(mod(p, rep));
}
```

For value noise, hash the **integer cell** with `mod(i, rep)` before fetching corners.

Useful maps from noise

```
float n = fbm(uv * 4.0);
float ridge = 1.0 - abs(n * 2.0 - 1.0);
float billow = abs(n * 2.0 - 1.0);
float terrace = floor(n * 5.0) / 5.0;
```

Random in a shader without UV (screen)

```
float rnd = hash12(gl_FragCoord.xy + vec2(u_time * 17.0, 0.0));
```

For dither, use a Bayer matrix or `hash12(gl_FragCoord.xy)` added at ~1/255.

What not to do

- `fract(sin(dot(uv, vec2(12.9898,78.233))) * 43758.5453)` as the only hash — it bands on some GPUs. Fine for a first lecture; replace later.
- Animating by adding `u_time` to **both** x and y of fbm without a direction — looks like TV static. Scroll: `fbm(uv + vec2(0.0, u_time * 0.1))`.

14 SDF and Ray Marching.md

14 — SDF and ray marching

Parent: 07 WebGL and Shader Snippets

Signed distance: **negative inside**, positive outside. Combine with `min` / `max`, not boolean mesh CSG.

Demo: `11-sdf-raymarch.html`.

Primitives (2D, for screens and masks)

```
float sdCircle(vec2 p, float r) { return length(p) - r; }

float sdBox(vec2 p, vec2 b) {
  vec2 d = abs(p) - b;
  return length(max(d, 0.0)) + min(max(d.x, d.y), 0.0);
}

float sdSegment(vec2 p, vec2 a, vec2 b) {
  vec2 pa = p - a, ba = b - a;
  float h = clamp(dot(pa, ba) / dot(ba, ba), 0.0, 1.0);
  return length(pa - ba * h);
}

float sdEquilateralTriangle(vec2 p, float r) {
  float k = sqrt(3.0);
  p.x = abs(p.x) - r;
  p.y = p.y + r / k;
  if (p.x + k * p.y > 0.0) p = vec2(p.x - k * p.y, -k * p.x - p.y) / 2.0;
  p.x -= clamp(p.x, -2.0 * r, 0.0);
  return -length(p) * sign(p.y);
}
```

Primitives (3D)

```
float sdSphere(vec3 p, float r) { return length(p) - r; }

float sdBox(vec3 p, vec3 b) {
  vec3 q = abs(p) - b;
  return length(max(q, 0.0)) + min(max(q.x, max(q.y, q.z)), 0.0);
}

float sdRoundBox(vec3 p, vec3 b, float r) { return sdBox(p, b) - r; }

float sdTorus(vec3 p, vec2 t) {
  vec2 q = vec2(length(p.xz) - t.x, p.y);
  return length(q) - t.y;
}

float sdCapsule(vec3 p, vec3 a, vec3 b, float r) {
  vec3 pa = p - a, ba = b - a;
  float h = clamp(dot(pa, ba) / dot(ba, ba), 0.0, 1.0);
  return length(pa - ba * h) - r;
}

float sdPlane(vec3 p, vec3 n, float h) { return dot(p, n) + h; }

float sdCylinder(vec3 p, float r, float h) {
  vec2 d = abs(vec2(length(p.xz), p.y)) - vec2(r, h);
  return min(max(d.x, d.y), 0.0) + length(max(d, 0.0));
}
```

CSG

```
float opU(float a, float b) { return min(a, b); }
float opS(float a, float b) { return max(a, -b); }
float opI(float a, float b) { return max(a, b); }

float smin(float a, float b, float k) {
  float h = clamp(0.5 + 0.5 * (b - a) / k, 0.0, 1.0);
  return mix(b, a, h) - k * h * (1.0 - h);
}
```

Repeat and transform

```
vec3 opRep(vec3 p, vec3 c) { return mod(p + 0.5 * c, c) - 0.5 * c; }

vec3 rotY(vec3 p, float a) {
  float c = cos(a), s = sin(a);
  return vec3(c * p.x - s * p.z, p.y, s * p.x + c * p.z);
}
```

Scene and normal

```
float map(vec3 p) {
  float s = sdSphere(p - vec3(0.0, 0.6, 0.0), 0.6);
  float b = sdBox(p, vec3(1.4, 0.12, 1.4));
  return smin(s, b, 0.2);
}

vec3 calcNormal(vec3 p) {
  const vec2 e = vec2(1e-3, 0.0);
  return normalize(vec3(
    map(p + e.xyy) - map(p - e.xyy),
    map(p + e.yxy) - map(p - e.yxy),
    map(p + e.yyx) - map(p - e.yyx)
  ));
}
```

Sphere tracer

```
float march(vec3 ro, vec3 rd) {
  float t = 0.0;
  for (int i = 0; i < 80; i++) {
    float d = map(ro + rd * t);
    if (d < 0.001) return t;
    t += d;
    if (t > 40.0) break;
  }
  return -1.0;
}
```

Camera ray from UV in [−1,1] with aspect:

```
vec3 rd = normalize(vec3(uv * vec2(aspect, 1.0), -1.2));
```

Rotate `rd` by the same basis as the camera.

Soft shadow (from a point toward the light)

```
float softShadow(vec3 ro, vec3 rd, float mint, float maxt, float k) {
  float res = 1.0;
  float t = mint;
  for (int i = 0; i < 24; i++) {
    float h = map(ro + rd * t);
    res = min(res, k * h / t);
    t += clamp(h, 0.02, 0.2);
    if (res < 0.01 || t > maxt) break;
  }
  return clamp(res, 0.0, 1.0);
}
```

Ambient occlusion (cheap)

```
float ao(vec3 p, vec3 n) {
  float occ = 0.0;
  float sca = 1.0;
  for (int i = 0; i < 5; i++) {
    float h = 0.01 + 0.12 * float(i) / 4.0;
    float d = map(p + n * h);
    occ += (h - d) * sca;
    sca *= 0.95;
  }
  return clamp(1.0 - 3.0 * occ, 0.0, 1.0);
}
```

2D outline from an SDF

```
float d = sdCircle(uv, 0.4);
float fill = 1.0 - smoothstep(0.0, 0.002, d);
float stroke = 1.0 - smoothstep(0.0, 0.004, abs(d));
vec3 col = vec3(0.1);
col = mix(col, vec3(0.91, 0.31, 0.22), fill);
col = mix(col, vec3(1.0), stroke);
```

Limits to tell students

Ray marching is **O(pixels × steps × scene cost)**. A 4K screen with 128 steps and a heavy fbm map will melt a laptop. Cap steps, cap resolution, or march a small FBO.

15 Postprocess.md

15 — Postprocess snippets

Parent: 07 WebGL and Shader Snippets

Draw the scene to an FBO, then draw a fullscreen triangle sampling that texture. Demo: `13-framebuffer-post.html`.

Fullscreen vertex (one triangle)

```
#version 300 es
const vec2 v[3] = vec2[3](vec2(-1.0, -1.0), vec2(3.0, -1.0), vec2(-1.0, 3.0));
out vec2 v_uv;
void main() {
  vec2 p = v[gl_VertexID];
  v_uv = p * 0.5 + 0.5;
  gl_Position = vec4(p, 0.0, 1.0);
}
```

No VBO required in WebGL2 if you use `gl_VertexID`. The helper also has `GL.fullscreenVerts()` if you want an attribute.

Vignette

```
float vig = v_uv.x * v_uv.y * (1.0 - v_uv.x) * (1.0 - v_uv.y);
vig = pow(vig * 16.0, 0.4);
color *= vig;
```

Chromatic aberration

```
vec2 d = (v_uv - 0.5) * u_amount;
vec3 c;
c.r = texture(u_tex, v_uv + d).r;
c.g = texture(u_tex, v_uv).g;
c.b = texture(u_tex, v_uv - d).b;
```

Film grain

```
float g = hash12(v_uv * u_res + u_time) - 0.5;
color += g * 0.04;
```

Contrast / saturation / lift

```
color = (color - 0.5) * u_contrast + 0.5;
float luma = dot(color, vec3(0.2126, 0.7152, 0.0722));
color = mix(vec3(luma), color, u_sat);
color += u_lift;
```

Tone map (Reinhard, ACES-lite)

```
vec3 reinhard(vec3 x) { return x / (1.0 + x); }

vec3 aces(vec3 x) {
  const float a = 2.51, b = 0.03, c = 2.43, d = 0.59, e = 0.14;
  return clamp((x * (a * x + b)) / (x * (c * x + d) + e), 0.0, 1.0);
}
```

Use these **before** `toSRGB` if the scene is HDR-ish (bloom, strong lights).

Bright pass + blur (bloom-lite)

Bright pass:

```
float b = max(max(c.r, c.g), c.b);
vec3 bright = c * smoothstep(u_thresh, u_thresh + 0.2, b);
```

9-tap box (cheap):

```
vec3 blur9(sampler2D tex, vec2 uv, vec2 px) {
  vec3 s = vec3(0.0);
  for (int y = -1; y <= 1; y++)
  for (int x = -1; x <= 1; x++)
    s += texture(tex, uv + vec2(x, y) * px).rgb;
  return s / 9.0;
}
```

A real bloom is downsample → separable Gaussian → add. Teach the 9-tap first.

FXAA-lite (luma edges)

```
float luma(vec3 c) { return dot(c, vec3(0.299, 0.587, 0.114)); }
vec2 px = 1.0 / u_res;
float l = luma(texture(u_tex, v_uv).rgb);
float lL = luma(texture(u_tex, v_uv + vec2(-px.x, 0)).rgb);
float lR = luma(texture(u_tex, v_uv + vec2( px.x, 0)).rgb);
float lD = luma(texture(u_tex, v_uv + vec2(0, -px.y)).rgb);
float lU = luma(texture(u_tex, v_uv + vec2(0,  px.y)).rgb);
vec2 dir = vec2(lL - lR, lD - lU);
dir = clamp(dir * 8.0, -1.0, 1.0) * px;
vec3 c = 0.5 * (
  texture(u_tex, v_uv + dir).rgb +
  texture(u_tex, v_uv - dir).rgb);
```

This is a teaching blur-along-edge, not NVIDIA FXAA 3.11.

Invert / posterize / kernel

```
color = 1.0 - color;
color = floor(color * 5.0) / 5.0;
```

Sharpen:

```
vec3 c = texture(u_tex, v_uv).rgb * 5.0
       - texture(u_tex, v_uv + vec2(px.x, 0)).rgb
       - texture(u_tex, v_uv - vec2(px.x, 0)).rgb
       - texture(u_tex, v_uv + vec2(0, px.y)).rgb
       - texture(u_tex, v_uv - vec2(0, px.y)).rgb;
```

Depth of field (circle of confusion from depth texture)

```
float z = texture(u_depth, v_uv).r;
float coc = clamp(abs(z - u_focus) * u_scale, 0.0, 1.0);
vec3 sharp = texture(u_tex, v_uv).rgb;
vec3 blur = blur9(u_tex, v_uv, px * 3.0);
color = mix(sharp, blur, coc);
```

Needs a **linear** depth if you stored `gl_FragCoord.z`; raw NDC depth is nonlinear. For class, store view-space `z` in a color attachment instead.

Copy pass (identity)

Always keep an identity post shader. When the picture is wrong, bind it. If identity is wrong, the FBO is wrong, not the FX.

16 Effects.md

16 — Effect snippets

Parent: 07 WebGL and Shader Snippets

Small visual recipes. Each should be one lab, not a stack.

Dissolve

```
float n = valueNoise(v_uv * 8.0);
float d = n - u_cutoff;
if (d < 0.0) discard;
float edge = 1.0 - smoothstep(0.0, u_edge, d);
vec3 color = mix(albedo, u_edgeColor, edge);
```

Demo: `19-dissolve.html`. Animate `u_cutoff` from −0.1 to 1.1.

Checker / grid (no texture)

```
vec2 c = floor(v_uv * u_cells);
float chk = mod(c.x + c.y, 2.0);
vec2 g = abs(fract(v_uv * u_cells) - 0.5);
float line = 1.0 - smoothstep(0.45, 0.5, max(g.x, g.y));
```

Triplanar mapping

```
vec3 n = abs(normalize(v_worldN));
n = pow(n, vec3(u_sharp));
n /= n.x + n.y + n.z + 1e-5;
vec3 x = texture(u_tex, v_worldPos.zy * u_scale).rgb;
vec3 y = texture(u_tex, v_worldPos.xz * u_scale).rgb;
vec3 z = texture(u_tex, v_worldPos.xy * u_scale).rgb;
vec3 albedo = x * n.x + y * n.y + z * n.z;
```

Use on terrain and CSG where UVs are painful.

Parallax (steep, teaching)

```
float h = texture(u_height, v_uv).r;
vec2 offset = v_viewTS.xy / (v_viewTS.z + 0.42) * (h * u_scale);
vec2 uv = v_uv - offset;
```

`v_viewTS` is the view vector in tangent space. Easy to break; show a brick texture.

Water (sum of sines)

```
float wave(vec2 p, vec2 dir, float freq, float amp, float speed) {
  return sin(dot(p, dir) * freq + u_time * speed) * amp;
}
float h =
  wave(xz, normalize(vec2(1.0, 0.3)), 4.0, 0.04, 1.6) +
  wave(xz, normalize(vec2(-0.4, 1.0)), 7.0, 0.02, 2.1);
```

Normal from height field:

```
float e = 0.05;
float hx = height(xz + vec2(e, 0.0)) - height(xz - vec2(e, 0.0));
float hz = height(xz + vec2(0.0, e)) - height(xz - vec2(0.0, e));
vec3 N = normalize(vec3(-hx, 2.0 * e, -hz));
```

Demo: `20-water.html`.

Fire / lava

```
float n = fbm(vec2(uv.x * 2.0, uv.y * 3.0 - u_time * 0.4));
n += 0.5 * fbm(vec2(uv.x * 6.0, uv.y * 8.0 - u_time));
float f = smoothstep(0.2, 0.9, n * (1.0 - uv.y));
vec3 col = mix(vec3(0.1, 0.0, 0.0), vec3(1.0, 0.55, 0.05), f);
col = mix(col, vec3(1.0, 0.95, 0.6), pow(f, 4.0));
```

Sky + sun

```
vec3 rd = normalize(v_worldPos); // or camera ray
vec3 sky = mix(u_horizon, u_zenith, pow(max(rd.y, 0.0), 0.4));
float sun = pow(max(dot(rd, u_sunDir), 0.0), 256.0);
sky += vec3(1.0, 0.85, 0.5) * sun;
```

Demo: `21-sky.html`.

Hatching (luma → stripes)

```
float l = dot(color, vec3(0.3, 0.6, 0.1));
float h = sin((v_uv.x + v_uv.y) * 80.0);
float ink = smoothstep(0.0, 0.2, l - 0.15 - h * 0.08);
```

Matcap (see 12 Lighting)

Procedural matcap if you have no image:

```
vec2 m = vn.xy * 0.5 + 0.5;
float d = length(m - 0.5);
vec3 col = mix(vec3(0.15, 0.18, 0.22), vec3(0.9, 0.92, 0.95), 1.0 - d);
col += pow(max(1.0 - length(m - vec2(0.35, 0.65)), 0.0), 8.0);
```

Barycentric wireframe

```
float e = min(min(v_bary.x, v_bary.y), v_bary.z);
float w = fwidth(e);
float line = 1.0 - smoothstep(0.0, w * 1.15, e);
vec3 color = mix(albedo, u_lineColor, line);
```

Demo: `18-barycentric-wire.html`.

Vertex color / height color

```
vec3 col = mix(u_sand, u_grass, smoothstep(-0.2, 0.4, v_worldPos.y));
col = mix(col, u_snow, smoothstep(0.7, 1.1, v_worldPos.y));
```

Dithered transparency (no sort)

```
float a = texture(u_tex, v_uv).a;
if (a < hash12(gl_FragCoord.xy)) discard;
```

Useful for foliage. No blend state.

Polar / kaleido UV

```
vec2 p = v_uv * 2.0 - 1.0;
float r = length(p);
float a = atan(p.y, p.x);
a = mod(a, 6.28318 / u_slices);
p = vec2(cos(a), sin(a)) * r;
```

Screen-door / Bayer (4×4)

```
float bayer4(vec2 fc) {
  int x = int(mod(fc.x, 4.0));
  int y = int(mod(fc.y, 4.0));
  int m[16];
  // 0,8,2,10 / 12,4,14,6 / 3,11,1,9 / 15,7,13,5
  m[0]=0;m[1]=8;m[2]=2;m[3]=10;
  m[4]=12;m[5]=4;m[6]=14;m[7]=6;
  m[8]=3;m[9]=11;m[10]=1;m[11]=9;
  m[12]=15;m[13]=7;m[14]=13;m[15]=5;
  return float(m[y * 4 + x]) / 16.0;
}
```

GLSL ES 3.00 allows the array. Use it to dither a cutoff.

Motion (UV scroll)

```
vec2 uv = v_uv + vec2(u_time * 0.03, 0.0);
```

Wrap with `fract` if the texture is REPEAT.

17 Particles and GPGPU.md

17 — Particles, instancing, GPGPU

Parent: 07 WebGL and Shader Snippets

Demos: `14-instancing.html`, `15-particles.html`, `23-gpgpu-pingpong.html`.

Instancing (WebGL2)

Per-instance attribute: translation (or a whole mat4 as four vec4s).

```
gl.bindBuffer(gl.ARRAY_BUFFER, instBuf);
gl.enableVertexAttribArray(3);
gl.vertexAttribPointer(3, 3, gl.FLOAT, false, 0, 0);
gl.vertexAttribDivisor(3, 1);
gl.drawArraysInstanced(gl.TRIANGLES, 0, 36, N);
```

Vertex:

```
layout(location = 3) in vec3 a_instancePos;
vec3 pos = a_position * u_scale + a_instancePos;
```

`divisor 0` = per vertex; `1` = per instance.

Instance color:

```
layout(location = 4) in vec3 a_instanceColor;
out vec3 v_color;
```

Point particles (CPU-updated)

```
gl.enable(gl.BLEND);
gl.blendFunc(gl.SRC_ALPHA, gl.ONE); // additive
gl.depthMask(false);
gl.drawArrays(gl.POINTS, 0, n);
gl.depthMask(true);
gl.disable(gl.BLEND);
```

Vertex:

```
gl_PointSize = u_size / gl_Position.w;
```

Fragment: discard outside a circle (see 11 Vertex and Fragment).

Reset blend and depthMask or the next opaque pass is wrong.

Transform feedback (WebGL2, optional lecture)

```
gl.beginTransformFeedback(gl.POINTS);
gl.enable(gl.RASTERIZER_DISCARD);
gl.drawArrays(gl.POINTS, 0, n);
gl.disable(gl.RASTERIZER_DISCARD);
gl.endTransformFeedback();
```

Bind a buffer to `TRANSFORM_FEEDBACK_BUFFER`. Varyings must be listed with `gl.transformFeedbackVaryings` **before** link. Easy to get wrong; ping-pong textures are a gentler Course 12.

GPGPU ping-pong (RGBA float positions)

Two textures: `read` and `write`. A fullscreen pass samples `read` and writes `write`. Then swap.

Position texture: `xy = pos.xy`, `zw = vel.xy` (2D) or two textures for 3D.

```
vec4 s = texture(u_state, v_uv);
vec2 pos = s.xy;
vec2 vel = s.zw;
vel += u_acc * u_dt;
pos += vel * u_dt;
if (pos.x < -1.0 || pos.x > 1.0) vel.x *= -0.8;
if (pos.y < -1.0 || pos.y > 1.0) vel.y *= -0.8;
outColor = vec4(pos, vel);
```

Create textures with `RGBA16F` or `RGBA32F` and `gl.texImage2D(..., gl.RGBA, gl.FLOAT, null)`. Check `EXT_color_buffer_float` for rendering to them.

Then a **render** pass: one point per texel, vertex shader samples the state texture using `gl_VertexID` → UV.

```
int id = gl_VertexID;
vec2 uv = (vec2(id % u_w, id / u_w) + 0.5) / vec2(u_w, u_h);
vec4 s = texture(u_state, uv);
gl_Position = vec4(s.xy, 0.0, 1.0);
gl_PointSize = 3.0;
```

Curl / flow (2D)

```
float n1 = valueNoise(pos * 2.0 + vec2(0.0, u_time * 0.1));
float n2 = valueNoise(pos * 2.0 + vec2(5.2, u_time * 0.1));
vec2 acc = vec2(n2 - 0.5, n1 - 0.5) * 0.4;
```

Soft particles (intersect scene depth)

```
float sceneZ = texture(u_sceneDepth, gl_FragCoord.xy / u_res).r;
float fade = clamp((sceneZ - gl_FragCoord.z) * u_soft, 0.0, 1.0);
outColor.a *= fade;
```

Needs a depth texture from the opaque pass.

Budget talk (say this in lab)

Count	Approach
< 2k	CPU update + POINTS
2k–50k	instancing or TF
50k–1M	GPGPU ping-pong
more	compute (WebGPU) or give up on the laptop

Common bugs

- Forgot `vertexAttribDivisor` → every instance is the same
- Float FBO incomplete → no `EXT_color_buffer_float`
- Additive blend left on
- `PointSize` clamped; check `ALIASED_POINT_SIZE_RANGE`
- Sampling the texture you are writing (no ping-pong)